

MediCore Interview Preparation Guide

Detailed project explanation, architecture, interview questions, answers, and revision notes.

React | Express | MongoDB | Node.js | JWT | OTP | PDFKit | Brevo | Render

Use this PDF for interview preparation. Focus on explaining the business problem, the role-based workflows, security decisions, and the technical implementation.

1. Project Overview

MediCore is a full-stack Hospital Management System built with React, Express, MongoDB, and Node.js. It provides separate dashboards for patients, doctors, and admins. The project includes secure authentication, email OTP verification, doctor approval, appointments, lab services, medical records, billing, emergency handling, notifications, Excel import, report emails, and professional PDF generation.

The best interview angle is that MediCore is not just a CRUD app. It implements real healthcare workflows: unverified users cannot access dashboards, doctors need admin approval, admins can import Excel records, and reports/invoices can be generated as PDFs and sent through email.

2. Tech Stack

Frontend

React 18, Vite, React Router HashRouter, Tailwind CSS, Radix UI, Framer Motion, TanStack Query, Lucide icons.

Backend

Node.js, Express, MongoDB, Mongoose, JWT, bcrypt, PDFKit, XLSX, Multer, Cloudinary, Brevo API.

Database

MongoDB collections modeled with Mongoose schemas: User, Doctor, Patient, Appointment, Billing, Record, OTP, Emergency, Notification, Review, Department, Payment.

Deployment

Frontend as a static site and backend as a Node service. HashRouter prevents Render refresh Not Found errors.

3. Architecture

The React frontend calls Express APIs under /api. Express routes validate requests and use middleware for JWT authentication and admin-only access. Mongoose models handle persistence. Reusable services handle OTP generation, Brevo email templates, PDF generation, Cloudinary uploads, and Excel import/export.

A normal request flow is: user action in React -> API call with token -> Express route -> protect middleware -> business logic/service -> MongoDB -> JSON/PDF/email response.

4. Role-Based Workflows

Patient

Signs up, verifies email OTP, books appointments and lab services, views records/reports/billing, submits emergency data, manages settings.

Doctor

Signs up with professional details, verifies email, waits for admin approval, manages appointments/patients/consultations/reports/schedule/emergency cases.

Admin

Uses secret keyword, verifies email, manages users/doctors/patients/appointments/billing/reports/imports/emergencies/settings and approves doctors.

5. Authentication and Security

Passwords are hashed with bcrypt before saving. Login returns a JWT. Protected APIs require Authorization: Bearer <token>. The backend protect middleware checks token validity, blocked status, email verification, and doctor approval. Admin-only APIs also require admin role.

OTP flow: registration creates an unverified user, server generates a 6-digit OTP, stores a protected OTP with expiry, sends email through Brevo, user enters OTP in the verification popup, server verifies it, marks account verified, and redirects to dashboard.

- bcrypt hashes passwords before saving.
- JWT protects API requests.
- OTP verifies email ownership.
- Doctor approval protects doctor-only tools.
- Blocked users are rejected by middleware.
- Admin routes require adminOnly middleware.

6. Main API Sections

Auth

/auth/register, /auth/login, /auth/verify-otp, /auth/resend-otp, /auth/forgot-password, /auth/reset-password, /auth/me

Users

/users, /users/:id/block, /users/:id

Doctors

/doctors, /doctors/:id/approve, /doctors/:id/reject, /doctors/:id/schedule

Appointments

/appointments, /appointments/my-appointments, /appointments/:id

Records

/records, /records/patient/:patientId, /records/:id

Billing

/billing/services, /billing, /billing/:id/invoice, /billing/:id/pay

Reports

/reports/generate-*, /reports/email/*, /reports/import/*, /reports/export/*

Emergency

/emergency, /emergency/:id/assign, /emergency/:id/status, /emergency/stats

Other

/dashboard/stats, /notifications, /reviews, /departments, /payments, /upload, /health

7. Database Models

User

name, email, password, role, avatar, phone, gender, dateOfBirth, isVerified, status, approvalStatus, settings.

Appointment

patient, patientId, doctor, doctorId, department, date, time, status, type, notes, symptoms, services.

Billing

invoiceId, patient, doctor, service, services, amount, paid, status, source, date, dueDate, payment details.

OTP

email, OTP hash, type, expiry, attempts, createdAt. Used for email verification and password reset.

Record

Medical record data such as diagnosis, prescription, lab report, discharge summary, patient and doctor references.

Emergency

Emergency case data, severity/status, assignment, notes, patient information.

8. Reports, Emails, Uploads and Imports

- PDFKit generates prescriptions, lab reports, discharge summaries, and invoices.
- Brevo sends OTP, account status, report, and notification emails.
- XLSX handles admin Excel imports and exports.
- Multer and Cloudinary support profile photos and medical document uploads.
- Billing stores service, amount, paid, status, invoice, and payment details.

9. Deployment Notes

The frontend is deployed as a static site and the backend as a Node service. The app uses HashRouter so refresh works on Render. VITE_API_URL points the frontend to the backend /api URL, while CLIENT_URL configures allowed CORS origin and email dashboard links.

10. Resume Bullets

- Built a full-stack hospital management system using React, Express, MongoDB, and Node.js with patient, doctor, and admin dashboards.
- Implemented secure authentication with JWT, bcrypt, email OTP verification, admin secret keyword, blocked-account checks, and doctor approval workflow.
- Developed professional PDF generation and email delivery for prescriptions, lab reports, discharge summaries, and billing invoices using PDFKit and Brevo.
- Added Excel import/export workflows for patients, doctors, and billing records using XLSX and backend validation.
- Improved Render deployment reliability by switching to HashRouter and hash-based dashboard links.

11. Common Interview Questions and Answers

Q. Tell me about your project.

A. MediCore is a full-stack hospital management system with patient, doctor, and admin dashboards. It includes OTP authentication, doctor approval, appointments, lab services, records, billing, emergencies, reports, PDF documents, Excel import, uploads, settings, and email notifications.

Q. What problem does it solve?

A. It centralizes hospital operations. Patients can manage care, doctors can manage consultations and reports, and admins can control users, approvals, imports, billing, and emergency cases from one platform.

Q. Which technologies did you use?

A. React and Vite for frontend, Tailwind and Radix for UI, Express and Node for APIs, MongoDB and Mongoose for database, JWT and bcrypt for auth, PDFKit for PDFs, XLSX for import, Brevo for email, Cloudinary for uploads.

Q. How does authentication work?

A. Users register, password is hashed, OTP is sent to email, account stays unverified until OTP is correct. After login/verification, server returns JWT and frontend sends it as Bearer token to protected APIs.

Q. Why do doctors need admin approval?

A. OTP only verifies email ownership. Doctors handle medical workflows, so admin approval verifies trust and credentials before enabling doctor dashboard access.

Q. How do you protect routes?

A. Frontend uses ProtectedRoute and RoleRoute. Backend uses protect middleware and adminOnly middleware. Backend is the real security layer because it verifies JWT, role, status, verification, and approval.

Q. What caused the Brevo email error?

A. Brevo requires htmlContent or textContent. The fix was to ensure every email payload includes safe text and HTML content using reusable email templates.

Q. How are PDFs generated?

A. The backend uses PDFKit to generate prescriptions, lab reports, discharge summaries, and invoices as PDF buffers. These can be downloaded or attached to Brevo emails.

Q. How does Excel import work?

A. Admin uploads .xlsx or .xls. Multer receives the file, XLSX parses rows, validation checks required fields, and valid records are inserted or updated in MongoDB.

Q. Why did Render show Not Found on refresh?

A. With BrowserRouter, refreshing /dashboard asks Render for a physical /dashboard file. HashRouter stores routes after #, so Render only serves / and React handles the route.

Q. How do notifications work?

A. Notifications are stored in MongoDB and exposed through APIs for listing, unread count, mark read, clear all, and delete. The dashboard displays them to users.

Q. What would you improve next?

A. Add automated tests, request validation middleware, audit logs, rate limiting, refresh tokens, background queues for email/PDF jobs, route-level lazy loading, and stronger upload scanning.

Q. How would you scale the app?

A. Use managed MongoDB indexes, paginate large lists, split frontend bundles, move email/PDF to background workers, use cloud storage for files, add monitoring and logs, and cache stable data.

Q. What was your biggest challenge?

A. Making authentication smooth and secure for three roles. Patients need OTP only, doctors need OTP plus admin approval, and admins need OTP plus secret keyword. I solved it with route guards and backend middleware.

Q. How would you explain this in one minute?

A. MediCore is a production-style MERN hospital system with secure role-based authentication, patient/doctor/admin dashboards, report PDFs, lab services, billing, emergencies, Excel import, uploads, and email notifications. I focused on real workflows, security, and deployment reliability.

12. Tough Follow-up Questions

Q. Why localStorage for token?

A. It is simple for this project. In a production healthcare app I would consider httpOnly secure cookies with CSRF protection or a short-lived access token plus refresh token approach.

Q. How prevent OTP brute force?

A. Rate limit by email and IP, limit attempts per OTP, expire OTP quickly, add resend cooldown, temporarily lock repeated failures, and log suspicious activity.

Q. How protect medical data?

A. Strict RBAC, backend authorization, audit logs, HTTPS, secure file storage, minimal API responses, encryption where needed, and compliance review for healthcare data rules.

Q. How improve frontend performance?

A. Lazy-load dashboard routes, split heavy report/chart pages, reduce bundle size, paginate lists, memoize expensive components, and cache API data with TanStack Query.

Q. How add tests?

A. Backend API tests for auth/OTP/reports/imports, frontend tests for route guards and forms, and E2E tests for signup, OTP, login, doctor approval, and report generation.

13. Demo Flow

- Open home page and explain role-based CTA behavior.
- Show signup and OTP verification popup.
- Explain admin secret keyword and doctor approval.
- Show patient lab services, billing, reports, and settings.
- Explain PDF generation and email delivery.
- Mention Render refresh fix using HashRouter.

14. Final Revision Checklist

- Memorize the 30-second and 2-minute project explanation.
- Explain patient, doctor, and admin flows clearly.
- Explain JWT, bcrypt, OTP, role checks, and doctor approval.
- Know why HashRouter fixed Render refresh errors.
- Know why Brevo requires htmlContent or textContent.
- Practice explaining PDFKit, XLSX, Cloudinary, and MongoDB models.
- Prepare one bug story and one improvement story.